

```
import nltk
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from nltk.corpus import movie_reviews, stopwords
from sklearn.feature_extraction.text import TfidfVectorizer
```

```
nltk.download('movie_reviews')
nltk.download('stopwords')
```

```
[nltk_data] Downloading package movie_reviews to /root/nltk_data...
[nltk_data]   Unzipping corpora/movie_reviews.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
True
```

```
# Load file IDs
positive_ids = movie_reviews.fileids('pos')
negative_ids = movie_reviews.fileids('neg')

# Load raw text
positive_reviews = [
    movie_reviews.raw(fileid) for fileid in positive_ids
]

negative_reviews = [
    movie_reviews.raw(fileid) for fileid in negative_ids
]

print(f"Positive reviews: {len(positive_reviews)}")
print(f"Negative reviews: {len(negative_reviews)}")
```

```
Positive reviews: 1000
Negative reviews: 1000
```

```
stop_words = stopwords.words('english')
```

```
vectorizer_pos = TfidfVectorizer(
    stop_words=stop_words,
    max_df=0.9,
    min_df=5
)
```

```
vectorizer_neg = TfidfVectorizer(
    stop_words=stop_words,
    max_df=0.9,
    min_df=5
)
```

```
# TF-IDF matrices
tfidf_pos = vectorizer_pos.fit_transform(positive_reviews)
tfidf_neg = vectorizer_neg.fit_transform(negative_reviews)
```

```
def get_top_terms(tfidf_matrix, feature_names, top_n=15):
    mean_tfidf = np.asarray(tfidf_matrix.mean(axis=0)).ravel()
    top_indices = mean_tfidf.argsort()[::-1][:top_n]
    return pd.DataFrame({
        'term': feature_names[top_indices],
        'mean_tfidf': mean_tfidf[top_indices]
    })
```

```

return pd.DataFrame({
    'term': feature_names[top_indices],
    'tfidf_score': mean_tfidf[top_indices]
})

```

```

top_pos_terms = get_top_terms(
    tfidf_pos,
    np.array(vectorizer_pos.get_feature_names_out()),
    top_n=15
)

top_neg_terms = get_top_terms(
    tfidf_neg,
    np.array(vectorizer_neg.get_feature_names_out()),
    top_n=15
)

print("Top Positive Terms:")
print(top_pos_terms)

print("\nTop Negative Terms:")
print(top_neg_terms)

```

Top Positive Terms:

	term	tfidf_score
0	film	0.058488
1	movie	0.037038
2	one	0.034708
3	like	0.024537
4	story	0.020077
5	good	0.019837
6	life	0.019037
7	time	0.018796
8	also	0.018507
9	well	0.018435
10	even	0.017684
11	character	0.017596
12	characters	0.017375
13	much	0.016817
14	would	0.016804

Top Negative Terms:

	term	tfidf_score
0	film	0.056384
1	movie	0.046414
2	one	0.036460
3	like	0.027130
4	even	0.022020
5	good	0.020723
6	bad	0.019969
7	time	0.019770
8	would	0.019087
9	get	0.018896
10	story	0.018665
11	much	0.018333
12	plot	0.018001
13	character	0.017556
14	characters	0.017357

```

fig, axes = plt.subplots(1, 2, figsize=(16, 6))

# Positive reviews
axes[0].barh(
    top_pos_terms['term'],
    top_pos_terms['tfidf_score'],
    color='green'
)

```

```
axes[0].set_title("Top 15 TF-IDF Terms (Positive Reviews)")
axes[0].invert_yaxis()

# Negative reviews
axes[1].barh(
    top_neg_terms['term'],
    top_neg_terms['tfidf_score'],
    color='red'
)

axes[1].set_title("Top 15 TF-IDF Terms (Negative Reviews)")
axes[1].invert_yaxis()

plt.tight_layout()
plt.show()
```

